

개발기관	2025-XXXX-XX
배정연번	

개발이력	
최초 개발	2025-11-01
1차 업데이트	1.
2차 업데이트	2.
3차 업데이트	3.

「 AI 제조 기술 교육-25-XX 」

ChatGPT음성 비서 (Whisper + GPT +
gTTS) – PRD v1.0

제 출 일 :

소 속 :

성 명 :

※ 워크북 제출시 반드시 파일명을 ○○○○(제출년월일8자리)_○○○(성명).hwp로 하여
주십시오.(예시 : 20250128_홍길동.hwp)

개발일자 : 2025-11-01

ChatGPT음성 비서 (Whisper + GPT + gTTS) - PRD v1.0

1. 목적: Streamlit UI에서 음성을 입력받아 STT(Whisper) → GPT 질의응답 → TTS(gTTS)로 응답을 재생하는 음성 기반 챗봇을 구현한다.

1) 프로젝트 개요 (Overview)

- 프로젝트명: ChatGPT음성 비서 프로그램
- 핵심 가치: 손이 자유로운(Hands-free) 대화형 정보 검색 및 업무 보조
- 플랫폼/기술: Python 3.10+, Streamlit, OpenAI Whisper(STT), OpenAI GPT, gTTS(TTS), ffmpeg
- 대상 사용자: 음성 위주 상호작용이 필요한 일반 사용자, 강의/회의 메모 사용자, 접근성 보조 필요 사용자

2) 범위 (Scope)

- In-Scope
 - 마이크 녹음 → STT 변환 → GPT 답변 → TTS 재생 풀사이클
 - 대화 기록 유지(st.session_state) & 말풍선 UI
 - 사이드바: OpenAI API Key 입력, 모델 선택(gpt-4, gpt-3.5-turbo 등), 초기화 버튼
 - 생성된 음성 파일(mp3) 임시 저장 후 자동 삭제
- Out-of-Scope
 - 계정/결제, 멀티 사용자 서버 운영, 장기 저장형 데이터베이스
 - 네이티브 앱(iOS/Android) 패키징

3) 성공 지표 (Success Metrics)

- 기능성: 음성→텍스트 인식 정확도(한국어/영어) 체감 품질 만족도 80%+
- 성능: 단일 질의-응답 왕복 5초 내 평균 응답(네트워크에 따라 가변)

- 안정성: 30분 연속 사용 시 오류율 < 1%
 - UX: 최초 사용 3분 내 온보딩 완료(키 입력, 모델 선택, 첫 대화 성공)
-

4) 사용자 시나리오 (Scenarios)

4.1 기본 사용 흐름

Step 1: 애플리케이션 시작

- 사용자가 Streamlit 애플리케이션을 실행
- 사이드바에서 OpenAI API Key 입력
- 모델 및 언어 설정 선택

Step 2: 음성 입력

- 사용자가 마이크 버튼을 클릭하여 음성 녹음 시작
- 또는 오디오 파일을 업로드
- 녹음 완료 후 "변환" 버튼 클릭

Step 3: STT 처리

- Whisper가 음성을 텍스트로 변환
- 왼쪽 컬럼에 변환된 질문 텍스트 표시
- 로딩 상태 표시

Step 4: GPT 답변 생성

- 변환된 텍스트가 GPT API에 전달
- GPT가 답변 생성
- 오른쪽 컬럼에 답변 텍스트 표시

Step 5: TTS 음성 재생

- gTTS가 답변 텍스트를 음성으로 변환
- 자동으로 음성 재생
- 음성 파일 다운로드 옵션 제공

Step 6: 대화 기록 저장

- 질문-답변 쌍이 세션에 저장
 - 채팅 히스토리에 표시
-

4.2 예외 처리 시나리오

시나리오 1: API Key 미입력

- 사용자가 API Key를 입력하지 않은 경우
- 경고 메시지 표시: "OpenAI API Key를 입력해주세요"
- 기능 사용 불가 상태

시나리오 2: 음성 인식 실패

- Whisper가 음성을 인식하지 못한 경우
- 에러 메시지 표시: "음성을 인식할 수 없습니다. 다시 시도해주세요"
- 재시도 옵션 제공

시나리오 3: API 호출 실패

- OpenAI API 호출 중 오류 발생
- 에러 메시지 표시: "서버 오류가 발생했습니다. 잠시 후 다시 시도해주세요"
- 에러 로그 기록

시나리오 4: 음성 파일 생성 실패

- gTTS 음성 생성 중 오류 발생
 - 텍스트 답변은 표시하되, 음성은 재생하지 않음
 - 에러 메시지 표시
-

5) 기능 요구사항 (Functional Requirements)

ID	항목	상세
FR-1	STT	마이크 녹음 파일을 Whisper로 텍스트 변환 (ko/en 자동 감지). 실패 시 오류 메시지 제공.
FR-2	질의응답	변환 텍스트를 GPT에 전달해 요약/검색/설명 등 일반 질의응답 수행. 시스템 프롬프트로 간결/정확/근거 지시.
FR-3	TTS	GPT 답변 텍스트를 gTTS로 mp3 음성 생성 및 재생. 파일은 임시 저장 후 즉시 삭제 .
FR-4	UI	Streamlit 2-Column 채팅 UI (좌: 사용자, 우: 챗봇). 말풍선 스타일, 타임스탬프 표기.
FR-5	대화 기록	기 st.session_state["messages"]에 구조화 저장(role, content, audio_path?). 페이지 새로고침 시 유지.
FR-6	사이드바	API Key 입력, 모델 선택, 대화 초기화 버튼, 사용 가이드/주의사항 표시.
FR-7	오디오 처리	브라우저 녹음(WebRTC/Streamlit Recorder) → 서버에 업로드 → ffmpeg로 포맷 보정 후 STT.

6) 비기능 요구사항 (Non-Functional Requirements)

- **환경:** Python + Streamlit, ffmpeg 설치 필수
 - **보안:** API Key는 세션/환경변수로만 사용, 로그/프론트에 노출 금지
 - **성능:** 단일 요청 동시 10명 기준 평균 응답 5초 내(모델/네트워크에 따라 튜닝)
 - **호환:** Whisper, GPT, gTTS 동작 **확인** 및 라이브러리 버전 고정
 - **파일 수명주기:** mp3 임시 파일은 **재생 후 삭제**(예외 시 주기적 정리)
-

7) 시스템 아키텍처 (Architecture)

- [Browser/Mic] --(audio blob)--> [Streamlit App]

- | |
 - | STT(Whisper) <-----+-- [ffmpeg normalize]
 - | |
 - +--> UI(Chat bubbles) <-----+-- GPT(Inference via OpenAI API)
 - |
 - +-- TTS(gTTS -> mp3 -> autoplay) -> [Audio Player]
-

8) 데이터 모델 (Session Schema)

- session_state = {
 - "api_key": "*****",
 - "model": "gpt-4",
 - "messages": [
 - {"role": "user", "text": "라떼 칼로리?", "ts": 1690000000},
 - {"role": "assistant", "text": "라떼는 평균...", "ts": 1690000002, "audio_file": None}
 -]
 - }
-

9) UI/UX 요구 (Wireframe 요약)

- 상단: 앱 타이틀, 간단 설명
 - 좌측 사이드바: API Key 입력, 모델 선택, 초기화 버튼, 사용 가이드
 - 메인(2-Column):
 - 왼쪽: 사용자 말풍선(녹음 아이콘/텍스트)
 - 오른쪽: 챗봇 말풍선(텍스트 + ► 재생 버튼)
 - 하단 컨트롤: 🎤 녹음 시작/종료 버튼, 인식 언어 자동/수동 옵션
-

10) 외부 API 및 의존성

- OpenAI Whisper API(또는 로컬 Whisper 모델)
- OpenAI Chat Completions API(gpt-4 or gpt-3.5-turbo)

- gTTS(Google Text-to-Speech, 파이썬 라이브러리)
 - ffmpeg(오디오 포맷 보정)
 - streamlit-webrtc 또는 st-audiorec 등 녹음 컴포넌트
-

11) 에러 처리 & 예외 (Error Handling)

- API Key 미설정: 사이드바/모달 경고 → 키 입력 유도
 - 녹음 실패/무음: “음성이 감지되지 않았습니다” 재시도 버튼
 - STT 실패: Whisper 오류 메시지와 함께 **원본 오디오 저장 옵션**(디버깅용)
 - GPT 실패: 네트워크/한도 초과 메시지 + 재시도
 - TTS 실패: 텍스트 표시 유지, 음성만 생략(UX 일관성 유지)
 - 파일 누수: try/finally 또는 컨텍스트 매니저로 mp3 삭제 보장, 주기적 GC
-

12) 보안·개인정보

- API Key는 세션/환경변수만 사용, 로그/프론트 저장 금지
 - 업로드 오디오는 **로컬 임시 저장** 후 목적 처리/삭제
 - 개인정보/민감 정보 발화 시 **경고 배너** 노출(“민감정보 포함 자제”)
-

13) 운영/관측(옵션)

- 로깅: 요청 규모, 평균 응답시간, 실패율
 - 프롬프트 로그 비활성화(기본): 민감 데이터 최소 수집
 - 간단 대시보드: 최근 대화 수, 평균 STT/GPT/TTS 지연 시간
-

14) 수용 기준 (Acceptance Criteria)

- 마이크 녹음 → **STT 텍스트** 출력이 3초 내 완료(평균)
- GPT 답변이 **채팅 UI**에 표시되며, **gTTS 음성 재생** 성공
- 세션 기록 유지 및 초기화 버튼 정상 동작

- mp3 임시 파일 자동 삭제 확인(디렉터리 누수 없음)
- 사이드바에서 API Key/모델 선택이 즉시 반영
- 한국어/영어 양방향 STT 정상 인식(샘플 스크립트로 검증)

15) 테스트 계획 (Test Plan)

케이스	유형 예상 결과
TC-01 API Key 미입력	단위 키 경고 및 GPT 호출 불가
TC-02 무음 업로드	단위 STT 실패 안내 및 재시도
TC-03 정상 대화	통합 텍스트+음성 응답, mp3 삭제 OK
TC-04 기록 초기화	통합 messages 배열 초기화
TC-05 한국어/영어 혼합 통합 언어 자동 감지, 답변 품질 유지	
TC-06 파일 누수	품질 temp 디렉터리 누수 0

16) 마일스톤 (Milestones)

1. M1(일주일): 녹음 → STT → 텍스트 표시(기본 파이프라인)
 2. M2(2주): GPT 연동, 채팅 UI, 세션 상태/초기화
 3. M3(3주): gTTS 음성 재생 + 임시 파일 삭제, 에러 처리
 4. M4(4주): 성능/UX 다듬기, 문서화, 경량 배포
-

17) 리스크 & 대응

- 네트워크 지연/한도 → 재시도/지수 백오프, 간절 답변 유도
 - STT 정확도 편차 → 잡음 억제, 마이크 가이드, 프롬프트 후처리
 - 파일 누수 → 철저한 예외 처리와 주기적 청소 태스크
-

18) 향후 개선 (Future Improvements)

- Whisper 실시간 스트리밍 STT(중간 결과 표시)
 - TTS 목소리/언어 선택(여성/남성, ko/en)
 - 대화 로그 다운로드(txt/json/csv)
 - 멀티턴 문맥 강화 + 요약(긴 대화 자동 요약/키워드 추출)
 - 키워드 스팟팅("헤이 케이지엔" 호출어) 및 단축 명령
-

19) 개발 체크리스트 (Dev Checklist)

- ffmpeg 설치 확인
 - Whisper/GPT/gTTS 버전 고정 및 호환성 검증
 - 세션 상태 스키마 확정(messages)
 - 임시 파일 경로/권한/삭제 보장
 - 예외/로그 레벨 표준화(INFO/WARN/ERROR)
-

20) Streamlit 화면 흐름(간단)

1. 사이드바 설정 → API Key, 모델 선택, 초기화
 2. 메인 화면 → "녹음 시작" 버튼 클릭
 3. STT 결과 텍스트 확인
 4. GPT 답변 텍스트/음성 출력
 5. 대화 반복 또는 기록 초기화
-

부록 A. 권장 폴더 구조

- app/
 - └─ ui.py # Streamlit 레이아웃/상태
 - └─ stt.py # Whisper 래퍼
 - └─ llm.py # GPT 호출 래퍼
 - └─ tts.py # gTTS 생성/재생/삭제

- └─ audio.py # 녹음/ffmpeg 정규화
 - └─ config.py # 키/모델 설정
 - └─ utils.py # 공통(에러/시간/파일)
-

부록 B. 시스템 프롬프트(초안)

“당신은 음성 비서입니다. 사용자의 질문에 **간결하고 정확하게** 한국어로 답하세요.

필요 시 항목을 **번호 목록**으로 정리하고, 불확실한 정보는 추측하지 말고 ‘확인 필요’라고 답하세요.”